

Write a program to find Turnaround time and waiting time for FCFS and SJF FCFS.

```
#include <stdio.h>
struct Processes
{
    int ID, AT, BT, CT, TAT, WT, RT;
};
```

```
int main()
{
    int n;
```

```
    printf("Enter no. of processes");
    scanf("%d", &n);
```

```
    struct Processes p[n];
```

```
    for (int i = 0; i < n; i++)
```

```
    {
        p[i].ID = i + 1;
```

```
        printf("Process time of P%d:",
            p[i].ID);
```

```
        scanf("%d", &p[i].AT);
```

```
        printf("Burst time of P%d:",
```

```
            p[i].ID);
```

```
        scanf("%d", &p[i].BT);
```

```
    }
```

```
    for (int i = 0; i < n; i++)
```

```
    {
        for (int j = 0; j < n; j++)
```

```
        if (p[i].AT > p[j+1].AT)
```

```
            struct Processes temp = p[j];
```

```
            p[j] = p[j+1];
```

$p[i+1] = temp;$

int timePassed = 0, sumTAT = 0,
 sumWT = 0, sumRT = 0;
 for (int i = 0; i < n; i++) {

if (timePassed < $p[i].AT$) {
 timePassed = $p[i].AT$;
 }

$p[i].CT = timePassed + p[i].BT$;
 $p[i].TAT = p[i].CT - p[i].AT$;
 $p[i].WT = p[i].TAT - p[i].BT$;
 $p[i].RT = p[i].WT$;

timePassed += $p[i].BT$;
 sumTAT += $p[i].TAT$;
 sumWT += $p[i].WT$;
 sumRT += $p[i].RT$;
 }

printf("No. Process | TAT |
 BT | CT | TAT | WT |
 RT | n");

for (int i = 0; i < n; i++) {
 printf("%d | %d | %d | %d |
 %d | %d | %d | %d |
 %d | n", p[i].ID, p[i].BT,
 p[i].CT, p[i].TAT,
 p[i].WT, p[i].RT);
 }

point D ("In Average TAT : 90.20 In",
(float) Sum TAT (n));
point F ("In Average WT : 90.20 In",
(float) Sum WT In);
point E ("In Average RT : 90.20 In",
(float) Sum RT (n));

return 0;
4

DIP

Enter number of processes: 2
Arrival Time of P1: 0
Burst Time of P1: 3
Arrival Time of P2: 1
Burst Time of P2: 2

Process	AT	BT	CT	TAT	WT	RT
P1	0	3	3	3	0	0
P2	1	2	5	4	2	2

Average TAT : 3.50
Average WT : 1.00
Average RT : 1.00

SJF

(non-preemptive)

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i, j;
```

```
int bt[20], wt[20], tat[20], p[20];
```

```
float avg_wt = 0, avg_tat = 0;
```

```
printf("Enter number of processes:");  
scanf("%d", &n);
```

```
printf("Enter burst time for  
each process: \n");
```

```
for (i = 0; i < n; i++)
```

```
{ printf("Process : ", i+1);
```

```
scanf("%d", &bt[i]);
```

```
p[i] = i+1;
```

```
}
```

```
for (i = 0; i < n-1; i++)
```

```
{ for (j = i+1; j < n; j++)
```

```
{ if (bt[i] > bt[j])
```

```
{ temp = bt[i];
```

```
bt[i] = bt[j];
```

```
bt[j] = temp;
```

```
temp = p[i];
```

```
p[i] = p[j];
```

```
p[j] = temp;
```

```
}
```


2
21

wt[0] = 0;

for (i = 2; i < n; i++) {

wt[i] = wt[i-2] + b[i-2];

}

for (i = 0; i < n; i++) {

tat[i] = b[i] + wt[i];

}

printf("In Process \t Burst Time \t
Waiting Time \t Turnaround Time\n");

for (i = 0; i < n; i++) {

printf("%d \t %d \t %d \t
%d\n", p[i], b[i], wt[i],
tat[i]);

avg-wt += wt[i];

avg-tat += tat[i];

}

avg-wt /= n;

avg-tat /= n;

printf("In Average waiting Time:

%0.2f", avg-wt);

printf("In Average Turnaround
Time: %0.2f\n", avg-tat);

return 0;

}

Enter number of processes : 4

Enter burst time for each process

P1: 7

P2: 3

P3: 4

P4: 6

Process	Burst Time	Waiting Time	Turnaround Time
P2	3	0	3
P3	4	3	7
P4	6	2	13
P1	7	13	26

Average Waiting Time : 5.75
Average Turnaround Time : 10.75

Shrey
10/3

SRF (primitive)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Enter no. of process");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], rt[n], ht[n];
    int ct[n], tat[n], wt[n], r[n];

    int completed = 0, current_time = 0;
    float t-at = 0, t-wt = 0, t-rt = 0;

    for (i = 0; i < n; i++) {
        printf("Enter pid: ");
        scanf("%d", &pid[i]);
        printf("Enter AT: ");
        scanf("%d", &at[i]);
        printf("Enter BT: ");
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
        ht[i] = -1;
    }

    while (completed != n) {
        int min_index = -1;
        int min_remaining = INT_MAX;
        for (i = 0; i < n; i++) {
            if (at[i] <= current_time & rt[i] > 0) {
                if (ht[i] < min_remaining) {
                    min_remaining = rt[i];
                    min_index = i;
                }
            }
        }
        current_time = at[min_index];
        completed++;
        rt[min_index] = 0;
        ht[min_index] = ht[min_index] + current_time - at[min_index];
        tat[min_index] = current_time - at[min_index];
        wt[min_index] = current_time - at[min_index] - bt[min_index];
        r[min_index] = current_time - at[min_index] - bt[min_index] - rt[min_index];
    }
}
```

2
2
4
if (min_index == -1) {
2 current = time + t;
2

else {
if (ct[min_index] == -1)
rt[min_index] = current_time - cost
[min_index];
Print ("P " "rd ", pid [min_index])
rt = rt [min_index];
current = time + t;
if (ct - wt [min_index] == 0) {
completed ++;
ct [min_index] = current_time;
tot [min_index] = ct [min_index] -
at [min_index];
wt [min_index] = tot [min_index] -
bt [min_index];
total_tot += tot [min_index];
total_wt += wt [min_index];
total_rt += rt [min_index];
2
2
2

Print ("In\n PID\t AT\t BT\t
CT\t TAT\t utt\t 1\t 2\t")

for (i = 0; i < n; i++) {


```
printf("%d\t %d\t %d\t %d\t %d\n",  
    readIn);
```

4

```
printf("avg + AT = %d", total  
    + atVal);
```

```
printf("In avg WT = %d", total_wtVal);  
printf("In avg RT = %d", total_rthVal);
```

```
return 0;
```

24

018

Enter no. of process: 4

Enter pid: 1

AT: 6

BT: 8

PID: 2

AT: 1

BT: 4

PID: 3

AT: 2

BT: 2

PID: 4

AT: 3

BT: 1

PID	AT	BT	CT	TAI	WT	RT
			15	15	7	0
1	0	8	15	7	3	0
2	1	4	8	2	0	0
3	2	2	4	2	1	1
4	3	1	5	2		

avg TAT = 6.50

Avg WT = 2.75

Avg RT = 0.25

~~Q2~~
17/3/26

Priority (Preemptive)

Write a C program to simulate the following CPU scheduling algorithm to find turn around time.

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i, time = 0, completed = 0, pos;
```

```
int at[10], bt[10], pt[10], rt[10];
```

```
int ct[10], fct[10], wt[10];
```

```
int min;
```

```
float avg_tat = 0, avg_wt = 0;
```

```
printf("Enter no. of process");
```

```
scanf("%d", &n);
```

```
for (i = 0; i < n; i++)
```

```
{ printf("Process %d AT: ", i+1);
```

```
scanf("%d", &at[i]);
```

```
printf("Process %d BT: ", i+1);
```

```
scanf("%d", &bt[i]);
```

```
printf("Process %d Priority: ", i+1);
```

```
scanf("%d", &pr[i]);
```

```
rt[i] = bt[i];
```

```
}
```

```
while (completed < n)
```

```
{ min = 999;
```

```
pos = -1;
```

```
for (i = 0; i < n; i++)
```

```
{ if (at[i] <= time && rt[i] > 0 &&  
pr[i] < min) &
```

$\frac{SRT}{NP}$

```

    mem = arr[i];
    pos = i;
    2

```

$\%F$ (pos = -1) @ time(t);

```

else {
    rt[post]--;
    time++;
}

```

if $Cr[pass] = -0.5$
Completed ++;

at [pos] = time t

$$\text{Let } [p_{203}] = \alpha [p_{03}] - \alpha^2 [p_{02}]$$
$$wt[pos] = tot[pos] - bt[pos]$$

2
2
2

```
printf("\n PID\t AT\t BT\t  
PR\t TA\t WT\n");
```

for $(i=0; i < n; i++)$ {

```
printf("%d\t %d\t %d\t %d\t %d\n",
```

$$^{p+2}, a^{[i]}, b^{[i]}, \phi^{[i]}, t a^{[i]},$$

avg. fast + = fast[i];
avg. cost +

$$2 \text{ avg. wt} + 1 \text{ wt} = \text{wt} [11];$$

```
print ("Avg. TAT = %2f" %
      avg_tat / n);
```

point $f''(n)$ in $\text{Arg. } w$ $T \equiv T_0 \cdot 2 \in \mathbb{N}$,
 $\text{arg} - w \neq T(n)$.

return 0;
2

O/P:

Enter no. of process: 4

process 1 AT: 0

process 1 BT: 0.1

process 1 Priority: 2

process 2 AT: 2

process 2 BT: 4

process 2 Priority: 1

process 3 Priority: 5

process 3 AT: 4

process 3 BT: 1

process 4 AT: 5

process 4 AT: 5

process 4 BT: 4

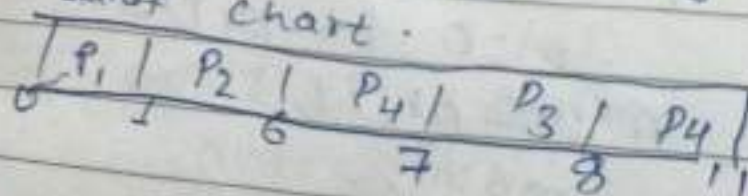
process 4 Priority: 2

PID	AT	BT	Priority	TAT	WT
1	0	1	2	0	0
2	2	4	1	4	0
3	4	1	5	4	3
4	5	4	2	6	2

Avg. TAT = 8.75

Avg. WT = 1.25

Gantt chart.



Priority Queue - Preemptive)

#include <stdio.h>

int main()

int n, i, j, pos;

int at[10], bt[10], pr[10];

int ct[10], tot[10], wt[10];

int time = 0, completed = 0, min;

float avg_tct = 0, avg_wt = 0;

printf("Enter no. of process:");

scanf("%d", &n);

for (i = 0; i < n; i++)

printf("AT for %d: ", i+1);

scanf("%d", &at[i]);

printf("BT for %d: ", i+1);

scanf("%d", &bt[i]);

printf("Priority: ");

scanf("%d", &pr[i]);

2

int done[10] = {0};

while (completed < n)

min = 999;

pos = -1;

for (i = 0; i < n; i++)

if (at[i] < time && done[i]

== 0 && pr[i] < min)

min = pr[i];

pos = i;

2


```

21
if (pos == -1)
    line++;
22
else {
    time += bt[pos];
    ct[pos] = time;
    done[pos] = 1;
    completed++;
}
23

```

```

for (i=0; i<n; i++) {
    tot[i] = ct[i] - at[i];
    wt[i] = tot[i] - bt[i];
}
24

```

```

printf("In PID \n TAT \n + BT \n\n");
printf("PR \n WT \n\n");
for (i=0; i<n; i++) {
    printf("%d\t %d\t (%d\t %d\t %d\t\n",
           pr[i], tot[i], wt[i], bt[i],
           pr[i], tot[i], wt[i], bt[i]);
}

```

```

avg-tot = tot[i];
avg-wt = wt[i];
25

```

```

printf("In Avg. TAT = %.2f", avg-tot);
printf("In Avg. WT = %.2f", avg-wt);

```

Q10

Enter no. of process : 4

AT	For	1	:	0
BT	For	1	:	2
priority	For	1	:	2
AT	For	2	:	4
BT	For	2	:	1
priority	For	2	:	1
AT	For	3	:	4
BT	For	3	:	1
priority	For	3	:	3
AT	For	4	:	8
BT	For	4	:	4
priority	For	4	:	2

PID	AT	BT	PR	TAT	WT	CT
1	0	7	2	2	0	7
2	2	4	1	4	0	6
3	4	1	3	7	6	11
4	5	4	2	5	1	10

$$\text{Avg TAT} = \frac{2+4+7+5}{4} = 4.5$$

$$\text{Avg WT} = \frac{0+0+6+1}{4} = 1.75$$

Gantt chart

1P1 1P2 1P3 1P4

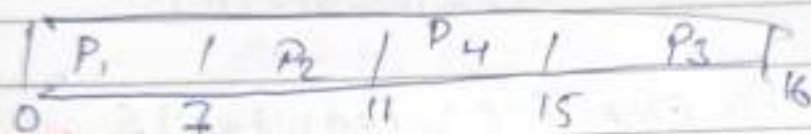
Result Table:

Pt D	AT	ST	PR	CT	TAT	WT
P ₁	0	7	2	7	7	0
P ₂	2	4	2	11	9	3
P ₃	4	1	3	16	12	11
P ₄	5	4	2	15	10	6

Average TAT = 9.5

Average WT = 5.5

Gantt Chart:



17/3/26

Round - Robin

```
#include <stdio.h>
int main()
{
    int n, tq;
```

```
printf("Enter number of processes:");
scanf("%d", &n);
```

```
int at[n], bt[n], rem_bt[n], ct[n],
fat[n], wt[n], rt[n];
int first_response[n];
```

```
for (int i = 0; i < n; i++) {
    printf("Enter AT and BT for\nP%d :", i+1);
    scanf("%d %d", &at[i], &bt[i]);
    rem_bt[i] = bt[i];
    first_response[i] = -1;
}
```

```
printf("Enter Time Quantum:");
scanf("%d", &tq);
```

```
int time = 0, completed = 0;
```

```
int gantt_p[100], gantt_t[100];
int idx = 0;
```

```
while (completed < n) {
    int done = 1;
```



```
for (int i = 0; i < n; i++) {  
    if (rem-bt[i] > 0 && at[i] <= time) {  
        done = 0;
```

```
    if (first-response[i] == -1) {  
        first-response[i] = time;  
        rt[i] = time - at[i];
```

21

```
        gant++ - p[idx] = i;  
        gant++ - t[idx] = time;  
        idx++;
```

```
    if (rem-bt[i] > tq) {
```

```
        time += tq;
```

```
        rem-bt[i] -= tq;
```

```
    } else {
```

```
        time += rem-bt[i];
```

```
        ct[i] = time;
```

```
        rem-bt[i] = 0;
```

```
        completed++;
```

22

23

24

```
    if (done) {
```

```
        time++;
```

25

26

```
    gant++ - t[idx] = time;
```

```
    printf("In Gantt Chart: \n");
```

```
    for (int i = 0; i < idx; i++) {
```

```
        printf(" - - - - - ");
```

27

```
printf("\n");
```

```
for (int i = 0; i < idx; i++) {
```

```
printf("p %d", gant - p[i] + 1);
```

```
}
```

```
printf("\n");
```

```
for (int i = 0; i <= idx; i++) {
```

```
printf(" %d - %d", gant + 1 + [i]);
```

```
}
```

```
printf("\n");
```

```
float total-wt = 0, total-tat = 0,  
total-rt = 0;
```

```
printf("\n Process | P | T | B | C |  
| TAT | WT | RT | \n");
```

```
for (int i = 0; i < n; i++) {
```

```
tat[i] = c[i] - a[i];
```

```
wt[i] = tat[i] - b[i];
```

```
total-tat += tat[i];
```

```
total-wt += wt[i];
```

```
total-rt += rt[i];
```

```
printf("P %d | T %d | B %d | C %d |  
%d | %d | %d | \n",  
i + 1, a[i], b[i], c[i],  
tat[i], wt[i], rt[i]);
```

```
}
```



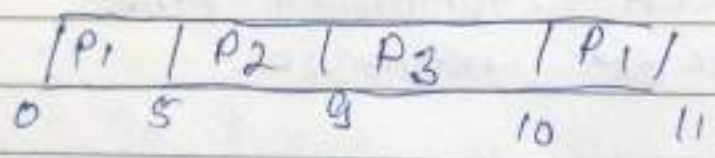
```
printf("\n Average TAT = %.2f",
        total_tat / n);
printf("\n Average WT = %.2f",
        total_wt / n);
printf("\n Average RT = %.2f",
        total_rt / n);

return 0;
```

Enter number of processes: 3
 Enter AT and BT for P1: 0 6
 Enter AT and BT for P2: 2 4
 Enter AT and BT for P3: 4 1

Enter Time Quantum: 5

Gantt Chart:



Process	AT	BT	CT	TAT	WT	RT
P ₁	0	6	11	11	5	0
P ₂	2	4	9	7	3	3
P ₃	4	1	10	6	5	3

Average TAT = 8.00
 Average WT = 4.33
 Average RT = 2.00

Enter Time Quantum: 1

$\begin{array}{|c|c|c|c|c|c|c|} \hline P_1 & P_2 & P_3 & P_1 & P_2 & P_1 & \\ \hline 0 & 2 & 4 & 5 & 7 & 9 & 11 \\ \hline \end{array}$

Process	AT	BT	CT	TAT	WT	RT
P ₁	0	6	11	11	5	0
P ₂	2	4	9	7	3	0
P ₃	4	1	5	1	0	0

Average TAT = 6.33

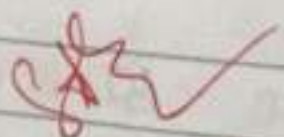
Average WT = 2.67

Average RT = 0.00

Conclusion

Smaller time Quantum gives better performance in terms of

- lower waiting time
- lower turnaround time
- faster response.


 24/2


```

#define MAX 100
#define MAX 100
#define MAX 100
#define MAX 100

```

```

#define MAX 100

```

```

typedef struct S
{
    int process-id;
    int arrival-time;
    int burst-time;
    int remaining-time;
    char type[10];
    int completion-time;
    int waiting-time;
    int turnaround-time;
} Process;

```

```

typedef struct S
{
    Process queue[MAX];
    int front, rear;
} Queue;

```

```

void initQueue(Queue *q) {
    q->front = q->rear = -1;
}

```

```

int isEmpty(Queue *q) {
    return q->front == -1;
}

```

```
void enqueue (Queue *q, Process p)
{
    if (q->rear == MAX-1) return;
    if (q->front == -1) q->front = 0;
    q->queue[t++] = p;
}

```

```
Process dequeue (Queue *q)
{
    Process p = q->queue[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    }
    else {
        q->front++;
    }
    return p;
}

```

```
int main()
{
    int n;
    printf("Enter number of processes\n");
    scanf("%d", &n);
}

```

```
Process process-list[MAX];
for (int i=0; i<n; i++) {
    printf("\nProcess %d\n", i);
    printf("Enter arrival time: ");
    scanf("%d", &process-list[i].arrival-time);
    printf("Enter burst time: ");
    scanf("%d", &process-list[i].burst-time);
}

```



```
printf("Enter type (0 = System,
        1 = User):");
```

```
int i;
```

```
scanf("%d", &t);
```

```
if (t == 0) strcpy(process-list[i].
    type, "System");
```

```
else strcpy(process-list[i]. type,
    "User");
```

```
process-list[i]. process-id = i;
```

```
process-list[i]. remaining-time =
```

```
process-list[i]. burst-time;
```

```
process-list[i]. completion-time = 0;
```

```
process-list[i]. waiting-time = 0;
```

```
process-list[i]. turnaround-time = 0;
```

```
}
```

```
for (int i=0; i<n-1; i++) {
```

```
    for (int j=i+1; j<n; j++) {
```

```
        if (process-list[i]. arrival-time >
```

```
            process-list[j]. arrival-time) {
```

```
            process temp = process-list[i];
```

```
            process-list[i] = process-list[j];
```

```
            process-list[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
Queue systemQueue, userQueue;
```

```
initQueue (& systemQueue);
```

```
initQueue (& userQueue);
```

```
int current-time = 0, completed = 0, i = 0;  
Process *current-process = NULL;
```

```
int gantt[MAX];  
int gantt-time[MAX];  
int gantt-index = 0;
```

```
while (completed < n) &
```

```
while (i < n && process-list[i].
```

```
arrival-time <= current-time) &
```

```
if (strcmp (process-list[i]. type,  
"System") == 0) &
```

```
enqueue (&system Queue,  
process-list[i]);
```

```
& else &
```

```
enqueue (&user Queue,  
process-list[i]);
```

```
&
```

```
i++;
```

```
&
```

```
if (current-process != NULL) &
```

```
if (strcmp (current-process->type,  
"User") == 0 && !is Empty  
(&system Queue)) &
```

```
enqueue (&user Queue, *current-  
process);
```

```
current-process = NULL;
```

```
&
```

```
&
```



```

if (current-process == NULL) {
    if (!isEmpty(SystemQueue)) {
        Process P = dequeue(SystemQueue);
        current-process = (Process *) malloc
            (sizeof(Process));
    }

```

```

    * current-process = P;
    2) else if (!isEmpty(UserQueue)) {

```

```

        Process P = dequeue(UserQueue);
        current-process = (Process *) malloc
            (sizeof(Process));

```

```

    * current-process = P;

```

```

    2) else {

```

```

        current-time++;

```

```

        continue;
    }

```

```

}

```

```

current-process->remaining-time--;
gantt[gantt-index] = current-process->
    process-id;
gantt-time[gantt-index] = current-time;
gantt-index++;
current-time++;

```

```

if (current-process->remaining-time
    == 0) {

```

```

    current-process->completion-time =
        current-time;

```

```

for (int k=0; k<n; k++) {
    if (process-list[k].process-id
        == current-process -> process-id)

```

```

    process-list[k].completion-time =
        current-process -> completion-time;
    process-list[k].turnaround-time =
        process-list[k].completion-time -
        process-list[k].arrival-time;

```

```

    process-list[k].waiting-time =
        process-list[k].turnaround-time -
        process-list[k].burst-time;

```

21

21

Completed ++;

Free (current-process);

current-process = NULL;

22

22

```

printf("\nID Type AT BT
CT WT TAT\n");

```

```

double totalWT=0, totalTAT=0;

```

```

for (int i=0; i<n; i++) {
    printf("%d %d %d %d %d %d\n",

```

```

        process-list[i].process-id,
        process-list[i].type,
        process-list[i].burst-time,

```



```

process = list + [i] * completion-time,
process = list + [i] * waiting-time,
process = list + [i] * turnaround-time);

```

```

totalWT += process-list[i] * waiting-time;
totalTAT += process-list[i] * turnaround-time;
}

```

```

printf("\n Gantt chart : \n");
for (int j = 0; j < gantt-index; j++) {
    printf("%d ", gantt[j]);
}

```

```

printf("\n");

```

```

for (int j = 0; j < gantt-index; j++) {
    printf("%d ", gantt-time[j]);
}

```

```

printf("%d \n", current-time);

```

```

return 0;

```

Input

Enter number of processes: 4

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = User): 1

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0 = System, 1 = User): 0

Process 3

Enter arrival time: 3

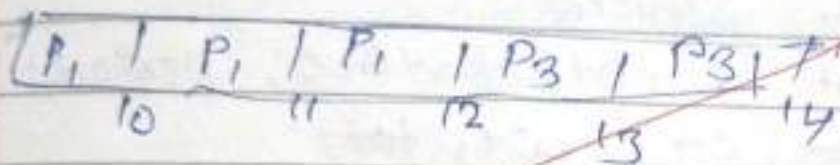
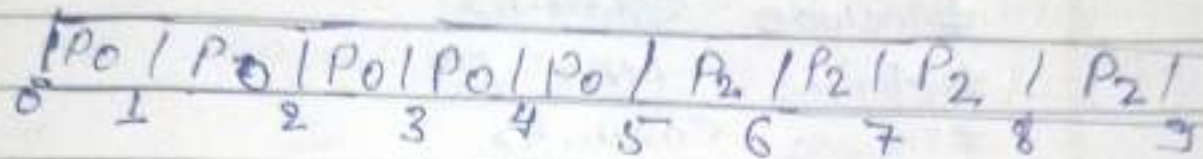
Enter burst time: 2

Enter type (0 = System, 1 = User): 1

id	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
2	System	1	4	9	4	8
1	User	2	3	12	7	10
3	User	3	2	14	9	11

Average Waiting Time: 5.00
 Average Turnaround Time: 8.50

Gantt chart:



Star
 24/2

write a C program to simulate
Real-Time CPU scheduling algorithms.
a) Rate-Monotonic
b) Earliest-deadline first.

```
#include <stdio.h>
#define MAX_ID
#include <math.h>

typedef struct
{
    int id, bt, deadline, period;
    int ct, wt, tat;
} Process;
```

```
void swap (Process *a, Process *b) {
    Process temp = *a;
    *a = *b;
    *b = temp;
}
```

2

```
void sortEDF (Process p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].deadline > p[j+1].
                deadline) {
                swap (&p[j], &p[j+1]);
            }
        }
    }
}
```

```
2 float calculateUtilization (Process p[], int n) {
    float U = 0.0;
```

2

```
    for (int i = 0; i < n; i++) {
        U += (float) p[i].bt / p[i].period;
    }
    return U;
}
```

2

```
2
```

```
2
```

```
void sortRMS (Process p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].period > p[j+1].period) {
                swap (&p[j], &p[j+1]);
            }
        }
    }
}
```


for (int i = 0; i < n-1; i++) {
 for (int j = 0; j < n-i-1; j++) {
 if (p[j].period > p[j+1].period) {
 swap(&p[j], &p[j+1]);
 }
 }
}

float calculateRMBound (int n) {
 return n * (pow(2, (float) 1/n) - 1);
}

void calculate (Process p[], int n) {
 int time = 0;
 for (int i = 0; i < n; i++) {
 time += p[i].bt;
 p[i].ct = time;
 p[i].tat = p[i].ct;
 p[i].wt = p[i].tat - p[i].bt;
 }
}

void ~~print~~EDF (Process p[], int n) {
 printf("ID\t BT\t Deadline\t CT\t WT\t
 TAT\n");
}

for (int i = 0; i < n; i++) {
 printf("ID\t BT\t Period\t AT\t
 %d\t %d\t %d\t %d\t %d\t
 %d\n", p[i].id, p[i].bt,
 p[i].deadline, p[i].ct, p[i].wt,
 p[i].tat);
}

void printRMSC (Process p[], int n) {
 printf("ID\t BT\t Period\t CT\t
 WT\t TAT\n");
}


```

for (int i = 0; i < n; i++) {
    printf("%d\t", p[i].id);
    printf("%d\t", p[i].burst);
    printf("%d\t", p[i].period);
    printf("%d\t", p[i].deadline);
    printf("%d\t", p[i].rt);
    printf("%d\t", p[i].util);
    printf("%d\t", p[i].tot);
}

```

```

int main() {
    int n;

```

```

    Process p[MAX], temp[MAX];

```

```

    printf("Enter number of processes\n");
    scanf("%d", &n);

```

```

// Step 0: calculate utilization
(p, n);

```

```

float bound =
    calculateRtBound(n);

```

```

printf("CPU utilization:
    %.2f\n", U);

```

```

printf("Rt Bound:
    %.4f\n", bound);

```

```

if (U <= bound)

```

```

    printf("Schedulable\n");
    else

```

```

    printf("Not
    schedulable\n");

```

```

    printf("\nEnter process details:\n");

```

```

    for (int i = 0; i < n; i++) {
        p[i].id = i;

```

```

        printf("Process %d: ", i);
        printf("Burst Time: ");

```

```

        scanf("%d", &p[i].burst);

```

```

        printf("Deadline (for EDF): ");
        scanf("%d", &p[i].deadline);

```

```

        printf("Period (for RMS): ");
        scanf("%d", &p[i].period);

```

```

    for (int i = 0; i < n; i++) temp[i] = p[i];
    SortEDF(temp, n);
    calculate(temp, n);

```

```

    printf("\n :: :: :: Earliest
    float U = calculateUtilization(n);
    printf("CPU Utilization: ");

```

```

    Deadline float

```


EDF) Scheduling $z == z \setminus n$;
print EDF(temp, n);

for (int i=0; i<n; i++) temp[i]=p[i];
sort RMS (temp, n);
calculate (temp, n);

printf("\n === Rate Monotonic
Scheduling CRMS | === \n");
print RMS (temp, n);

return 0;
}

Enter number of processes: 3

Enter process details:

Process 0:

Burst Time: 3

Deadline (for EDF): 10

Period (for RMS): 6

Process 1:

Burst Time: 2

Deadline (for EDF): 5

Period (for RMS): 4

Process 2:

Burst Time: 1

Deadline (for EDF): 8

Period (for RMS): 3

==== Earliest Deadline First (EDF)

Scheduling = Not

CPU Utilization: $\frac{1.33}{2.5} = 0.532$ Schedulable (Utilization ≤ 1)

ID	BT	Deadline	CT	WT	TAT
1	2	5	2	0	2
2	1	8	3	2	3
0	3	10	6	3	6

==== Rate Monotonic Scheduling (RMS):

==== 1.33

CPU Utilization: $\frac{1.33}{2.5} = 0.532$ RM Bound: 0.7798

ID	BT	Period	CT	WT	TAT
2	1	3	1	6	1
1	2	4	3	1	3
0	3	6	6	3	6

Not Schedulable (Utilization $>$ RM Bound)

The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0 = 1, S_1 = 0, S_2 = 0$. Find out how many times process P_0 will print "00". Assume the order of execution as P_0, P_1, P_2, P_0, P_1 .

Process P_0 :
 while (true) {
 wait(S_0);
 print("0");
 Signal(S_2);
 Signal(S_2);
 }

Process P_1 :
 wait(S_1);
 Signal(S_0);

Process P_2 :
 wait(S_2);
 Signal(S_0);

Initial	$S_0 = 1$	$S_1 = 0$	$S_2 = 0$	Remark
P_0	0	1	1	0
P_1	1	0	1	—
P_2	1	0	0	—
P_0	0	1	1	00
P_1	1	0	1	—

P_0 will print 0 two times

Consider two processes A and B. Two semaphores
 Variables S and T are used to
 synchronize the processes. S is
 initialized to 0 and T is initialized
 to 1. Processes are scheduled
 in the following order:
 AB ABBA AABA

What will be printed on the line

Process A:	Process B
while (1)	while (1)
S	wait(T);
wait(S)	print "1";
Print 'P';	Print "1";
Print 'P';	signal(S);
Signal(T);	

Initial	S = 0	T = 1	Remark
A	0	1	Process A % State
B	1	0	11
A	0	1	11pp
B	1	0	1pp11
B	1	0	1pp11
A	0	1	process B is blocked
A	0	1	11pp11pp
B	1	0	Process A is blocked
A	0	1	11pp11pp11
			11pp11pp11pp

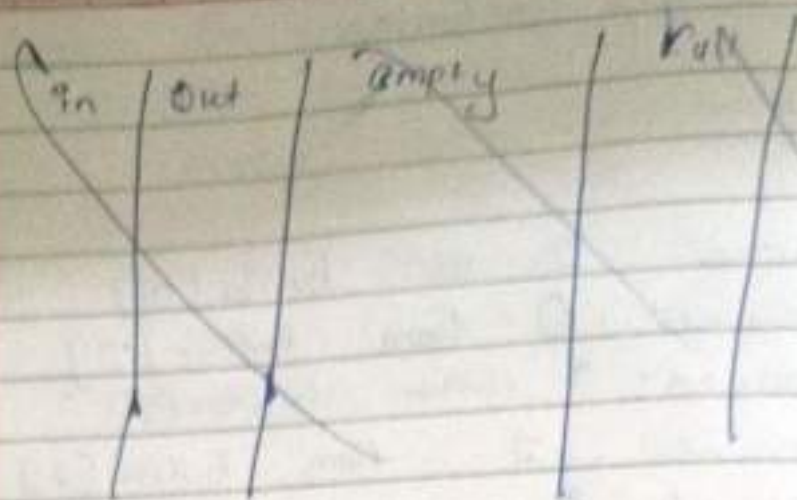
Producer & Consumer

Output

Produced : 0 at buffer [0]
 Consumed : 0 from buffer [0]
 Produced : 1 at buffer [1]
 Consumed : 1 from buffer [1]
 Produced : 2 at buffer [2]
 Consumed : 2 from buffer [2]
 Produced : 3 at buffer [3]
 Produced : 4 at buffer [4]
 Consumed : 3 from buffer [3]
 Consumed : 4 from buffer [4]

Tracing
 Buffer Size = 5

Step	Operation	Item	Buffer Index	Buffer State
1	Produced	0	buffer[0]	[0-----]
2	Consumed	0	buffer[0]	[------]
3	Produced	1	buffer[1]	[-1-----]
4	Consumed	1	buffer[1]	[-2-----]
5	Produced	2	buffer[2]	[-2-----]
6	Consumed	2	buffer[2]	[-3-----]
7	Produced	3	buffer[3]	[-3-----]
8	Produced	4	buffer[4]	[-34-----]
9	Consumed	3	buffer[3]	[-4-----]
10	Consumed	4	buffer[4]	[-4-----]



Tracing

Buffer size = 5

Buffer = [-, -, -, -, -]

in = 0, out = 0

empty = 5, full = 0, mutex = 1

Step	Operation	Buffer state	In	Out	empty	Full	mutex
1	producer inserts 0	[0, -, -, -, -]	1	0	4	1	0
2	Producer inserts 1	[0, 1, -, -, -]	2	0	3	2	0
3	Producer inserts 2	[0, 1, 2, -, -]	3	0	2	3	0
4	Producer inserts 3	[0, 1, 2, 3, -]	4	0	1	4	0
5	Producer inserts 4	[0, 1, 2, 3, 4]	0	0	0	5	0
6	Producer tries to insert (Blocked)	[0, 1, 2, 3, 4]	0	0	0	5	0
7	Consumer removes 0	[-, 1, 2, 3, 4]	0	1	1	4	1

8	Consumer removes 1	$[-, -, 2, 4]$	0	2	2	0	7
9	Consumer removes 2	$[-, -, 3, 4]$	0	3	3	2	7
10	Consumer removes 3	$[-, -, 4, 4]$	0	4	4	1	7
11	Consumer removes 4	$[-, -, 5, 4]$	0	0	5	0	1

SKZ
2/14

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define N 5
```

```
int buff[N], in=0, out=0, item=0, n;
sem_t, empty, full, mutex;
void * Producer (void * arg) {
    for (int i=0; i<n; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buff[in] = item;
        printf("Produced: %d at buffer\n", item, in);
        in = (in+1) % N;
    }
}
```

```

item + 1;
sem_post(&mutex);
sem_post(&full);

```

```

}
return NULL;

```

```

void consumer (void *args)

```

```

for (int i = 0; i < n; i++)

```

```

sem_wait(&full);

```

```

sem_wait(&mutex);

```

```

int var = buffer[out];

```

```

printf("consumed: %d from buffer\n", var, out);

```

```

out = (out + 1) % N;

```

```

sem_post(&mutex);

```

```

sem_post(&empty);

```

```

}

```

```

return NULL;

```

```

}

```

```

int main()

```

```

pthread_t, p, c;

```

```

printf("Enter number of items: ");

```

```

scanf("%d", &n);

```

```

sem_init(&empty, 0, N);

```

```

sem_init(&full, 0, 0);

```

```

sem_init(&mutex, 0, 1);

```

```

pthread_create(&c, NULL, consumer, NULL);

```

```

pthread_create(&p, NULL, producer, NULL);

```

```

pthread_join(p, NULL);

```

```

pthread_join(c, NULL);

```

```

return 0;

```

```

}

```


Dining - Philosopher

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define NUM_PHILOSOPHERS 5
#define ACTIVE_PHILOSOPHERS 4
#define MAX_CYCLES 3
```

```
pthread_mutex_t Forks [NUM_PHILOSOPHERS];
pthread_t philosophers [NUM_PHILOSOPHERS];
```

```
void * philosopher (void * num) {
    int id = *(int *) num;
```

```
    if (id >= ACTIVE_PHILOSOPHERS) {
        printf("Philosopher %d is inactive.\n",
            id);
        return NULL;
    }
```

20

```
    int left = id;
```

```
    int right = (id + 1) % NUM_PHILOSOPHERS;
```

```
    for (int i = 0; i < MAX_CYCLES; i++) {
```

```
        printf("Philosopher %d is thinking.\n", id);
```

```
        sleep(1);
```

if (id == 0) {

pthread_mutex_lock(&forks[id]);
printf("Philosopher %d picked
up left fork %d.\n", id, left);

pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked
up right fork %d.\n", id, right);

} else {

pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up
right fork %d.\n", id, right);

pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up
left fork %d.\n", id, left);

}
printf("Philosopher %d is eating.\n", id);
sleep(2);

pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down
forks %d and %d.\n",
id, left, right);
sleep(1);

}
printf("Philosopher %d has finished
.\n", id);
return 0;


```

int main() {
    int ids[NUM- PHILOSOPHERS];
    for (int i=0; i< NUM- PHILOSOPHERS; i++) {
        pthread_mutex_t forks[i], NULL;
    }

    for (int i=0; i< NUM- PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL,
            philosopher, &ids[i]);
    }

    for (int i=0; i< NUM- PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    printf("All philosophers have Dashed.\n");
    return 0;
}

```

OR

```

Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is is is thinking
Philosopher 4 is thinking
Philosopher 1 picked up right
Fork 2

```

```

Philosopher 1 picked up left Fork 1.
Philosopher 1 is eating.
Philosopher 0 picked up left Fork 0.
Philosopher 3 picked up right Fork 4.
Philosopher 3 picked up left Fork 3.

```

Philosopher 3 is eating.
 Philosopher 1 put down Forks
 2 and 3.
 Philosopher 2 is thinking.
 Philosopher 2 picked up left
 Fork 2.
 Philosopher 0 picked up right
 Fork 2.
 Philosopher 0 is eating.
 Philosopher 3 put down Forks
 3 and 4.
 Philosopher 3 is thinking.
 Philosopher 1 picked up left
 Fork 4.
 Philosopher 2 picked up
 Fork 3.
 Philosopher 2 is eating.
 Philosopher 0 put down Forks
 0 and 1.
 Philosopher 0 is thinking.
 Philosopher 1 picked up

28/9

#include <stdio.h>
#include <stdlib.h>

int main()

{ int p, r;

int i, j;

printf ("Enter number of processes");

scanf ("%d", &p);

printf ("Enter number of resource for each process");

scanf ("%d", &r);

int allocation [p][p], max [p][p], need [p][p];

int Available, [R], work[R];

bool finish [p];

int safeSequence [p];

printf ("\n Enter max Matrix : \n");

for (i=0; i<p; i++)

{ for (j=0; j<p; j++)

{ scanf ("%d", &max [i][j]);

}

printf ("Enter Available Resources: \n");

for (j=0; j<p; j++)

{ scanf ("%d", &Available [j]);

for (i=0; i<p; i++)

{ for (j=0; j<p; j++)

{ Need [i][j] = max [i][j] -

Allocation [i][j];

}

for $(j=0; j \leq R; j++)$
 work[j] = Available[j]

for $(j=0; j \leq P; j++)$
 Available[j] = false

the count = 0

while Count < P

 bool found = false;

 for $(j=0; j \leq P; j++)$

 if (Process[j] == false)

 bool canExecute = true;

 for $(j=0; j \leq R; j++)$

 if (Need[j][j] > work[j])

 canExecute = false;

 break;

 if (canExecute) &

 for $(j=0; j \leq R; j++)$

 work[j] += Allocation[j]

 if (work[j] == true;

 safe sequence Count++;

 found = true;

if (Count == false) &

 break;

for CS=0; r<P; r++) do

 P (max_held) = 10/100/2

 SAFE = false;

 break;

2

2

 P (safe) is

 printf ("In System 'P' is in Safe State
 \n" "Safe Sequence: ");

 for CS=0; r<P; r++) do

 printf ("P %d ", SafeSequence[i]);

 printf (" ");

2

else do

 printf ("In System 'P' is in
 UNSAFE STATE (Potential
 Deadlock) \n");

2

 return 0;

2

Q1P

Enter number of Process: 5

Enter number of Resource types: 4

2

Enter number of Resource types: 4

Enter Allocation, Matrix:

0	0	1	2
1	0	0	6
1	3	5	4
0	6	3	2
0	0	1	4

Enter Max Matrix

0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

Enter Available Resources:

1	5	2	0
---	---	---	---

~~System is in safe state.~~

for process:
for request[i]:

print("Enter process number making
request:");

scanf("%d", &process);

printf("Enter request vector: %d",

for (i = 0; i < m; i++)

scanf("%d", &request[i]);

for (i = 0; i < m; i++)

if (request[i] > need[process][i])

printf("Error: Request exceeds
maximum need. %d",

return 0;

2

for (i = 0; i < m; i++)

if (request[i] > available[i])

printf("Resources not
available. Access must wait %d",

return 0;

2

for (i = 0; i < m; i++)

available[i] -= request[i];

alloc[process][i] += request[i];

if (process < 0 || process > 100)

Q/D :

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0	1	0
5	0	2
2	1	1
0	0	2

Enter Maximum Demand Matrix:

7	4	5
0	2	0
6	0	0
0	1	-1
4	5	-1

Enter Available Resources:

2	3	0
---	---	---

Enter process number making

request: 4

Enter request vector:

3	3	0
---	---	---

Error: Request exceeds maximum need.

or,

Enter request vector

0	2	0
---	---	---

Deadlock Detection

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, m;

    int alloc[MAX], request[MAX][MAX];

    int work[MAX];
    int finish[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource
    types: ");
    scanf("%d", &m);

    printf("\nEnter Allocation Matrix: \n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter Request Matrix: \n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
}
```

```

    printf("\n Box Available Resource\n");
    for (int i=0; i<m; i++) {
        scanf("%d", &available[i]);
    }

```

21

```

    for (int i=0; i<m; i++)
        work[i] = available[i];

```

```

    for (int i=0; i<n; i++) {

```

```

        int flag=0;
        for (int j=0; j<m; j++) {
            if (alloc[j][i] == 0) {

```

```

                flag = 1;
                break;
            }

```

22

```

        }
        ans[i] = (flag == 0) ? 1 : 0;

```

23

```

        for (int k=0; k<n; k++) {
            for (int i=0; i<n; i++) {
                if (!ans[i]) {

```

```

                    int j;

```

```

                    for (j=0; j<m; j++) {
                        if (request[j][i] > work[j])
                            break;
                    }

```

24

```

                if (j == m) {

```

```

                    for (int x=0; x<m; x++)

```

```

                        work[x] += alloc[j][x];

```

```

                    ans[i] = 1;

```


21
24

24
24

int deadlock = 0;

printf("\n System Deadlock Analysis:
n");

for (int i = 0; i < n; i++) {

if (! finish[i]) {

if (deadlock == 0) {

printf("Deadlock detected\n");

printf("Processes causing

deadlock:");

24

printf("%d", i);

deadlock = 1;

24

24

if (! deadlock) {

printf("No deadlock detected.

All processes can finish\n");

24 else {

printf("\n These processes are
stuck and waiting for each
other.\n");

24

24

Enter number of processes: 5
Enter number of resource types: 3

Allocation matrix:

Enter
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Request Matrix

0 0 0
2 0 2
0 0 1
1 0 0
0 0 2

Enter Available Resources:

0 0 0

System Deadlock Analysis:
Deadlock Detected!

Processes causing deadlock: P₂ P₃ P₄

Those processes are stuck and waiting for each other.

$Req \leq Need$

↳

Req is available

Yes

$Need = Need - Req$

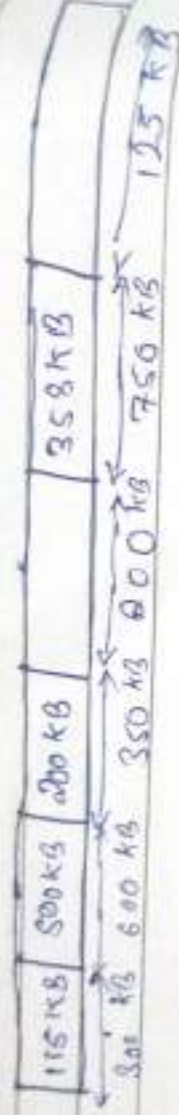
$Alloc = Alloc + Req$

$MA = MA - Req$

Safety algo { }

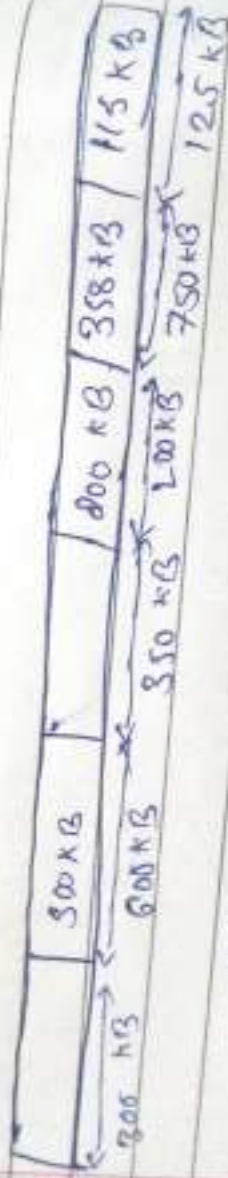
Memory partition : 300 KB, 600 KB, 350 KB, 200 KB, 200 KB
 250 KB (remainder)
 Process size : 115 KB, 500 KB, 358 KB, 200 KB, 200 KB
 375 KB (remainder)

Req: 134

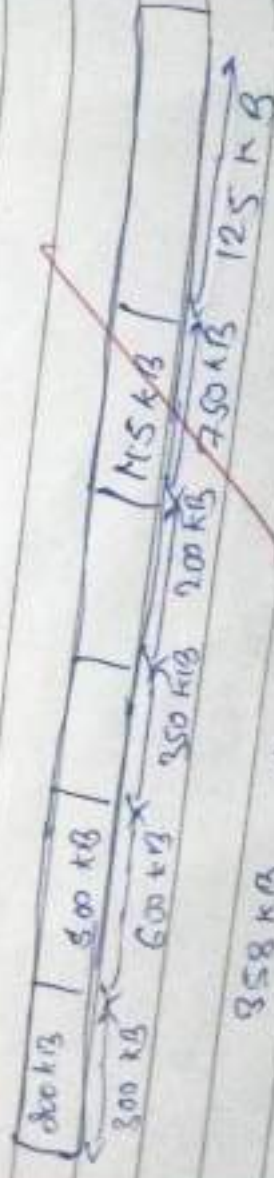


375 KB cannot be allocated with memory

Best Fit



375 KB cannot be allocated with memory
 Worst Fit



358 KB & 375 KB cannot be allocated with memory

Write a C program to simulate the following contiguous memory allocation techniques.

- Worst-Fit
- Best-Fit
- First-Fit

→ #include <stdio.h>

void FirstFit (int blockSize[], int m, int processSize[], int n)

{
 int allocation[n];

 for (int i=0; i<n; i++)

 allocation[i] = -1;

 for (int i=0; i<n; i++) {

 for (int j=0; j<m; j++) {

 if (blockSize[j] >= processSize[i])

 {

 allocation[i] = j;

 blockSize[j] = blockSize[j] - processSize[i];

 break;

 }

 }

 }

printf("\n --- First Fit --- \n");

printf("Process No. & Process Size & Block No. \n");

 for (int i=0; i<n; i++)

 {

 for (int i=0; i<n; i++)

 {

```

    printf("Process %d\n", i);
    processSize[i];

```

```

    if (allocation[i][2] == -1)
        printf("Not Allocated\n");
    else

```

2

2

```

    void bestFit (int blockSize[], int m,
    int processSize[], int n)

```

5

```

    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;

```

```

    for (int i = 0; i < n; i++)

```

5

```

    {
        int bestIdx = -1;
        for (int j = 0; j < m; j++)

```

3

```

    {
        if (blockSize[j] >= processSize[i])

```

```

        {
            if (bestIdx == -1 || blockSize[bestIdx] >
            blockSize[j])
                bestIdx = j;

```

2

2

```

        }
        if (bestIdx != -1)

```

3

```

            allocation[i] = bestIdx;

```



```

        blockSize[bestIdx] = processSize[i];
    }
}
    
```

```

    printf("\n - - Best fit - - \n");
    printf("Process No. | Process Size |  

    Block No. \n");
    
```

```

    for (int i = 0; i < n; i++)
    {
    
```

```

        printf("Process %d | %d |", i+1,
        processSize[i]);
    
```

```

        if (allocation[i] == -2)
    
```

```

            printf("Not Allocated\n");
        else
    
```

```

            printf("Not Allocated\n");
    }
}
    
```

2

```

void worstFit (int blockSize[], int m,
int processSize[], int n)
{
    int allocation[n];
    
```

```

    for (int i = 0; i < n; i++)
    {
        allocation[i] = -1;
    }
    
```

```

    for (int i = 0; i < n; i++)
    {
        allocation[i] = -1;
    }
    
```

```

    for (int i = 0; i < n; i++)
    {
        allocation[i] = -1;
    }
    
```

```

    for (int i = 0; i < n; i++)
    {
        allocation[i] = -1;
    }
    
```

```

    for (int i = 0; i < n; i++)
    {
        allocation[i] = -1;
    }
    
```

```

3  if (blockSize[i] >= processSize[j])
    3
    if (worstIdx == -1 || blockSize[i]
    > blockSize[worstIdx])
        worstIdx = i;
    2
    2
    if (worstIdx != -1)
        2
        allocation[i] = worstIdx;
        blockSize[worstIdx] = processSize[i];
    2
    2

```

```

printf("In ----- worst fit ----- \n");
printf("Access No. & Process Size &
Block No. \n");

```

```

for (int i = 0; i < n; i++)
    2
    printf("%d\t%d\t%d\t", i+1,
    processSize[i]);
    2
    if (allocation[i] != -1)
        printf("%d\t", allocation[i]+1);
    else
        printf("Not Allocated \n");
    2
    2

```



```
int main()
```

```
{
    int m, n;
```

```
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
```

```
    int blockSize[m];
```

```
    int block1[m], block2[m], block3[m];
```

```
    printf("Enter sizes of 1.d memory  
        blocks: ");
```

```
    for (int i=0; i<m; i++)
```

```
{
```

```
        scanf("%d", &blockSize[i]);
```

```
        block1[i] = blockSize[i];
```

```
        block2[i] = blockSize[i];
```

```
        block3[i] = blockSize[i];
```

```
}
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int processSize[n];
```

```
    printf("Enter sizes of 1.d processes: ");
```

```
    for (int i=0; i<n; i++)
```

```
        scanf("%d", &processSize[i]);
```

$BestFit(block\ 1, m, processSize, n);$
 $bestFit(block\ 2, m, processSize, n);$
 $consFit(block\ 3, m, processSize, n);$

return 0;

4

Output:

Enter number of memory blocks: 5

Enter sizes of 5 memory blocks:

100 500 200 300 800

Enter number of processes: 4

Enter sizes of 4 processes:

212 417 112 426

--- First Fit ---

Process No.

Process Size

Block No.

1	212	2
3	417	2
4	112	8
	426	2

--- Best Fit ---

Process No.

Process Size

Block No

1	212	Not Allocated
2	212	Block No
3	417	7
4	112	2
	426	3

--- 400g 174

Process No.	Process Size	Block No.
1	212	g
2	417	2
3	422	5
4	426	Not processed.

gh
1218

Write a C program to simulate page replacement algorithms.

Ans FIFO

↳ LRU

↳ Optimal

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int findOptimal Cnt pages[], int n,
```

```
int frame[], int frameCount, int index)
```

```
{ int farthest = index, pos = -1;
```

```
for (int i = 0; i < frameCount; i++) {
```

```
int j;
```

```
for (j = index; j < n; j++) {
```

```
if (frame[i] == pages[j]) {
```

```
if (j > farthest) {
```

```
farthest = j;
```

```
pos = j;
```

```
}
```

```
break;
```

```
}
```

```
}
```

```
if (j == n) return j;
```

```
}
```

```
return (pos == -1) ? 0 : pos;
```

```
}
```



```
we declare int time[10], pos=0;  
int min=999; int framecount; int i;  
for (int i=0; i<min; i++)
```

```
if (time[i] < min) {  
    min = time[i];  
}
```

```
pos = i;  
return pos;
```

```
void FIFO (int pages[], int n,
```

```
int framecount) {  
    int framecount, count = 0,  
    faults = 0;
```

```
for (int i=0; i<framecount; i++)  
    frame[i] = -1;
```

```
for (int i=0; i<n; i++) {  
    int found = 0;
```

```
    for (int j=0; j<framecount; j++)  
        if (frame[j] == page[i])  
            found = 1;  
    if (!found) {  
        faults++;  
        frame[j] = page[i];  
    }
```

```
if (i == framecount-1) {  
    cout << "FIFO  
    faults: " << faults << endl;
```

```
    cout << "Page  
    faults: " << faults << endl;
```

21

```
void LRU_Out_Pages [ ], int n, int HomeCount,
int Frame[HomeCount], time[HomeCount],
faults=0, counter=0;
for (int i=0; i<HomeCount; i++) Home[i]=i;
int found=0;
for (int j=0; j<HomeCount; j++) {
    counter++;
    time[j]=counter;
    found=1;
    break;
}
```

21

21

```
if (!found) {
    int pos=-1;
    for (int i=0; i<HomeCount; i++) {
        if (Home[i] == -2) {
            pos=i;
            break;
        }
    }
}
```

21

~~if (!found)~~

if (pos == -2)

pos=Find;

LRU (time, HomeCount);

Home[pos] = pages[i];

counter++;

time[pos] = counter;

faults++;

21

21

printf("LDV Page
faults);

21 optimal (int page[] , int n,

void out frameCount) {

int frame[frameCount];

for (int i = 0; i < frameCount; i++)

frame[i] = -1;

for (int i = 0; i < n; i++)

for (int j = 0; j < n; j++)

{

int found = 0;

for (int j = 0; j < frameCount; j++)

if (frame[j] == pages[i])

found = 1;

break;

}

if (!found)

int pos = -1;

for (int j = 0; j < frameCount; j++)

if (frame[j] == -1) pos = j;

break;

}

}

if (pos == -1)

pos = findDPFinal(pages, n,

frame, frameCount, i++);

frame[pos] = pages[i];

fault++;

}

printf ("Optimal pages faults: %d", frames);

20

int main() {

int n, frameCount;

printf ("Enter number of pages:");

scanf ("%d", &n);

int pages[n];

printf ("Enter the reference string:");

for (int i=0; i<n; i++)

scanf ("%d", &pages[i]);

printf ("Enter number of frames:");

scanf ("%d", &frameCount);

FIFO (pages, n, frameCount);

LRU (pages, n, frameCount);

Optimal (pages, n, frameCount);

return 0;

}

CIP

Enter number of pages: 20

Enter the reference string: 701103042

30321201701

Enter number of frames: 3

FIFO page faults: 15

LRU page faults: 12

Optimal page faults: 9

Page 2 → 2 - -

Page 1 → 2 0 -

Page 3 → 2 0 1

Page 0 → 0 0 1

Page 3 → 2 3 1

Page 0 → 2 3 0

Page 2 → 4 3 0

Page 2 → 4 2 0

Page 5 → 4 2 3

Page 0 → 0 2 3

Page 3 → 0 2 3

Page 2 → 0 2 3

Page 1 → 0 1 3

Page 2 → 0 1 2

Page 0 → 0 1 2

Page 1 → 0 1 2

Page 2 → 7 1 2

Page 0 → 7 0 2

Page 1 → 7 0 1

Total Page faults: 15

LRU Page Replacement

Page 2 → 2 - -

Page 0 → 2 0 - -

Page 2 → 7 0 1

Page 2 → 2 0 1

Page 0 → 2 0 1

page 3 $\rightarrow$ 200
 page 6 $\rightarrow$ 203
 page 4 $\rightarrow$ 403
 page 2 $\rightarrow$ 402
 page 3 $\rightarrow$ 432
 page 0 $\rightarrow$ 032
 page 3 $\rightarrow$ 032
 page 2 $\rightarrow$ 032
 page 1 $\rightarrow$ 132
 page 2 $\rightarrow$ 132
 page 0 $\rightarrow$ 102
 page 1 $\rightarrow$ 102
 page 7 $\rightarrow$ 107
 page 0 $\rightarrow$ 107
 page 1 $\rightarrow$ 107

Optimal Page Replacement

page 7 $\rightarrow$ 7 - -
 page 0 $\rightarrow$ 7 0 -
 page 1 $\rightarrow$ 7 0 1
 page 2 $\rightarrow$ 2 0 1
 page 0 $\rightarrow$ 2 0 1
 page 3 $\rightarrow$ 2 0 3
 page 0 $\rightarrow$ 2 0 3
 page 4 $\rightarrow$ 2 4 3
 page 2 $\rightarrow$ 2 4 3
 page 3 $\rightarrow$ 2 4 3
 page 0 $\rightarrow$ 2 0 3
 page 3 $\rightarrow$ 2 0 3
 page 2 $\rightarrow$ 2 0 3
 page 1 $\rightarrow$ 2 0 1

page 2 $\rightarrow$ 20 J
page 0 $\rightarrow$ 20 J
page 2 $\rightarrow$ 20 J
page 7 $\rightarrow$ 70 J
page 0 $\rightarrow$ 70 J
page 1 $\rightarrow$ 70 J

Total Page faults = 9

OK
OK